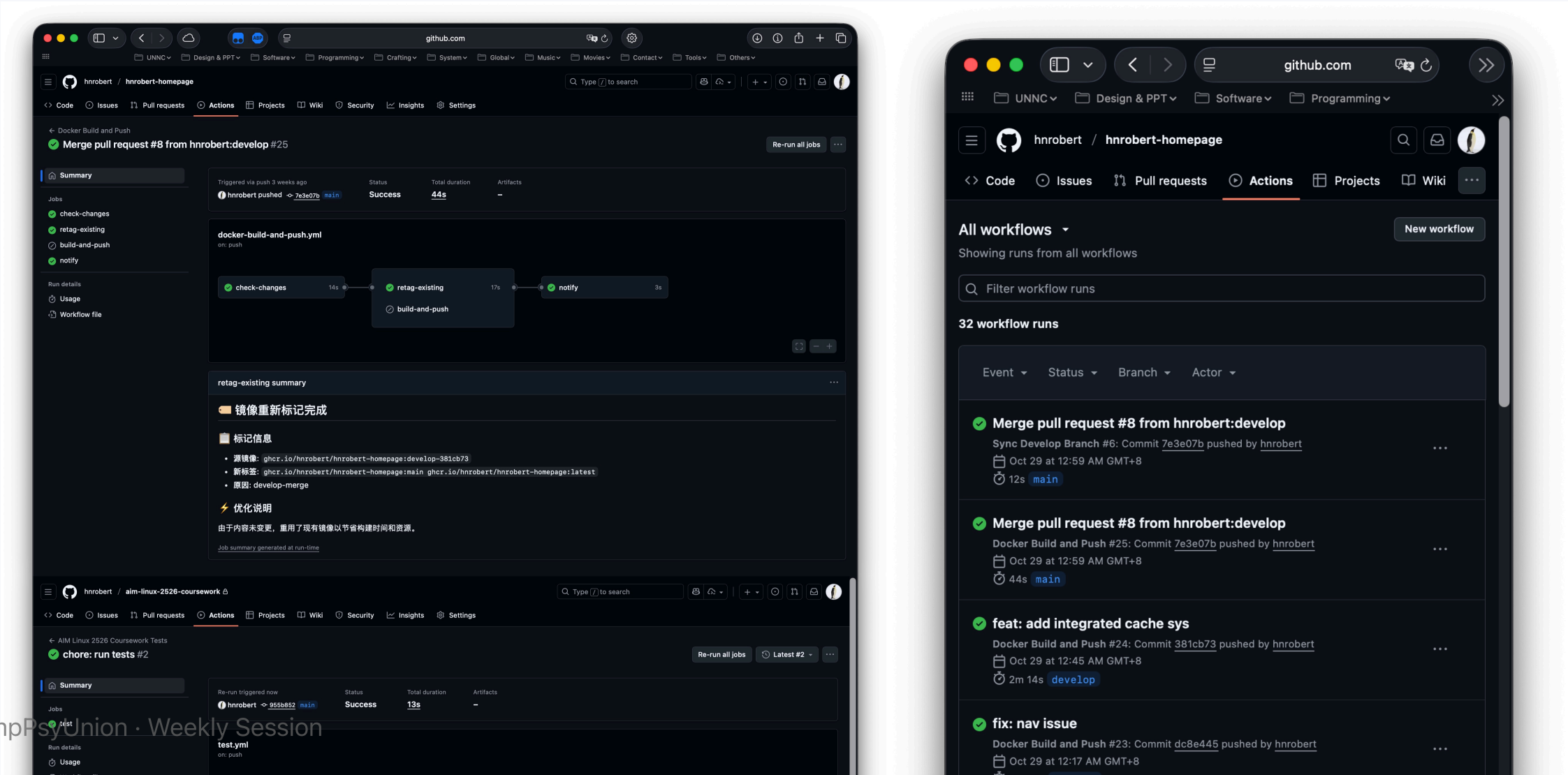


GitHub Actions — Automate your workflow



Agenda

- Welcome & Context
- What is GitHub Actions?
- Core Concepts
- Demo: Read a simple workflow
- Hands-On Practice
- Best Practices & Links
- Q&A

Welcome & Context

- Create automations for test, lint, package

What is GitHub Actions?

- CI/CD platform integrated with GitHub repos
- Automate your integration & development workflow on GitHub

Core Concepts

- CI: Continuous Integration automates testing & validation of code changes
- CD: Continuous Deployment automates releasing code to production

Workflows

- Automated processes defined by config files under this folder in your repo:

```
.github/workflows/
```

- Usually we define one workflow in one file, e.g., `put-an-elephant-in-a-fridge.yml`
- Each **workflow** is served as an independent automation
- **Jobs** under one **workflow** are triggered by the same **events**

Events

- When to run workflows. like:

```
on:  
  when-elephant-is-not-in-fridge:  
    # joke, this does not exist in real GitHub Actions
```

- Trigger workflows in GitHub:
 - `push`, `pull_request`, `release`, `schedule`, `workflow_dispatch`

Jobs

- Independent units of work that run in a runner

```
jobs:
  bring-elephant-indoor:
    runs-on: zoo
    # ...

  prepare-fridge:
    runs-on: indoor
    # ...

  send-the-elephant-into-the-fridge:
    runs-on: indoor
    # ...
```

Runners

- Where do jobs run

```
runs-on: campus
```

- For GitHub Action, this is virtual environments for jobs, also defined in this way:

```
runs-on: windows-latest
```

Steps

- Steps are `uses` or `run` commands, executed sequentially inside a job

```
steps:  
  - name: Open the fridge  
    uses: pre-defined/open-fridge-action@v1  
  
  - name: put the elephant in the fridge  
    run: mv elephant.zip /fridge  
  
  - name: Close the fridge  
    uses: pre-defined/close-fridge-action@v1
```

Matrix strategy

```
strategy:  
  matrix:  
    places: [room01, room02, room03]  
    fridges: [Xiaomi, SIEMENS]
```

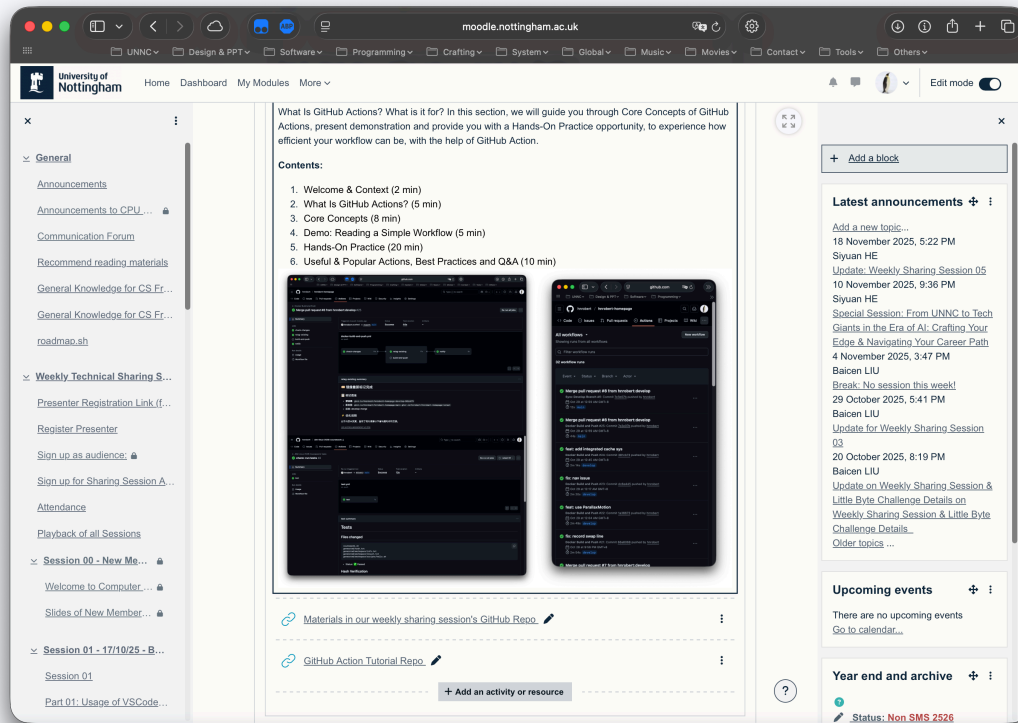
- Run jobs across rooms/brands of fridges combos (2x3=6 jobs here)

Actions marketplace

- Pre-built actions for common tasks (setup-python, cache, upload-artifact)

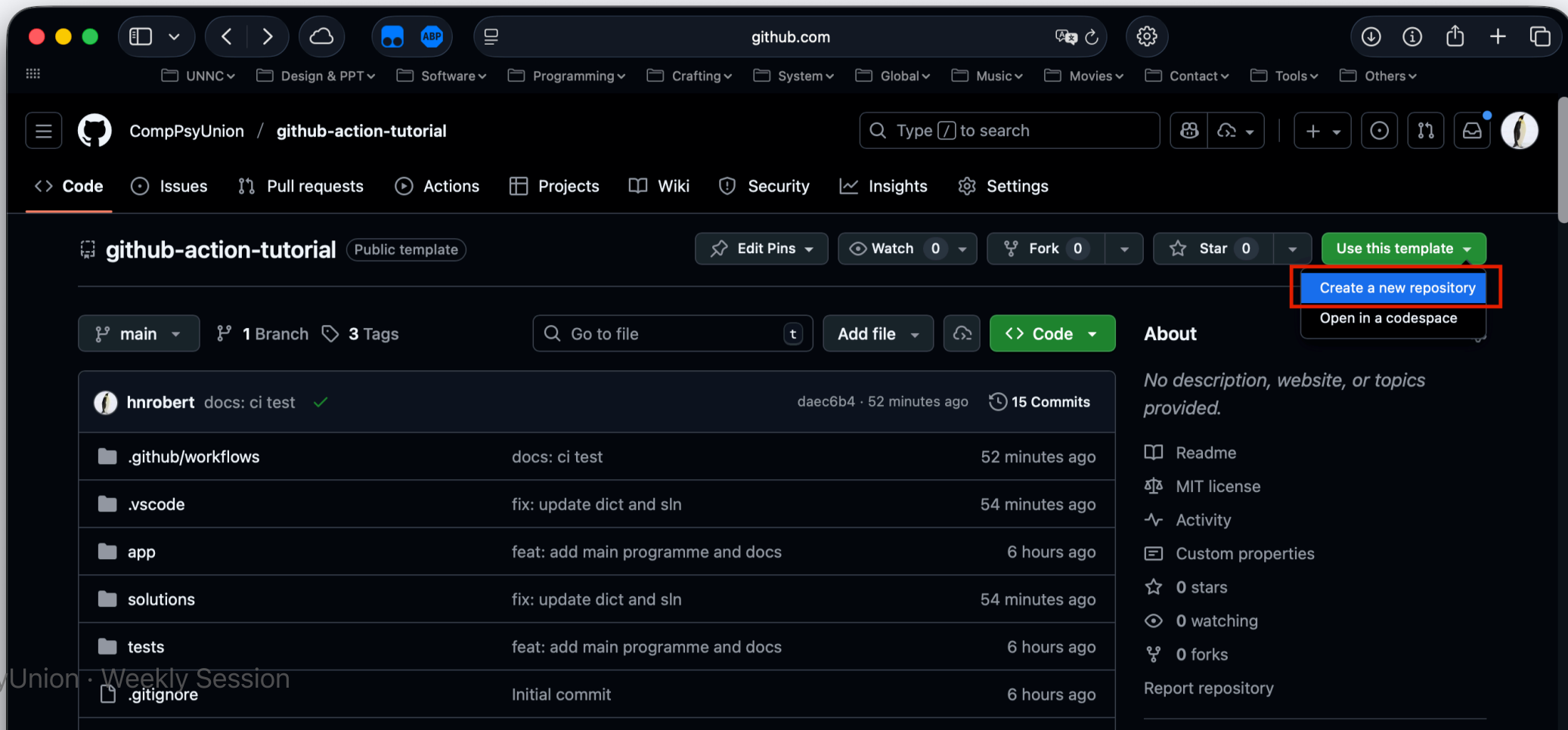
Demo: Reading a Simple Workflow (start)

- We'll open the tutorial repo and follow a short hands-on sequence.
- See links in Moodle or in the README to get started.



Quick Start — Create a new repo

- Use "Use this template" → Create a new repository



Quick Start — Name your repo

- Give a name & make it public (recommended)

The screenshot shows the GitHub 'Create a new repository' page. The page is divided into three sections: General, Configuration, and Jumpstart your project with Copilot (optional). The 'General' section is active and contains the following fields:

- Owner:** A dropdown menu showing 'hnrbert' with a red box around it and the annotation 'Select your own account here'.
- Repository name:** A text input field containing 'github-action-tutorial' with a red box around it and the annotation 'Give your own repo a name'. A green checkmark indicates the name is available.
- Description:** A text input field with a placeholder '0 / 350 characters'.
- Configuration:** A section with a 'Choose visibility' dropdown set to 'Public' with the annotation 'Keep it public'.
- Jumpstart your project with Copilot (optional):** A section with a 'Prompt' text area containing 'Describe what you want Copilot to build' and a character count '0 / 500 characters'.

At the bottom right, there is a green 'Create repository' button with a red box around it and the annotation 'Finally'.

Open Codespaces (optional)

- Create a codespace on `main` to explore workflows online

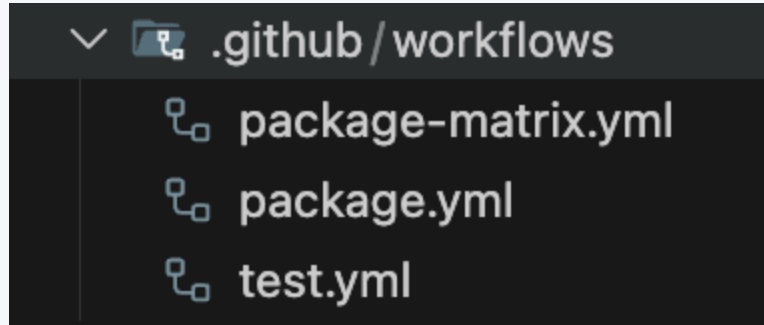
The screenshot shows the GitHub repository page for 'github-action-tutorial' by user 'hnrobert'. The repository is public and generated from 'CompPsyUnion/github-action-tutorial'. The 'Code' button is highlighted with a red box and a red number '1'. Below the repository name, the 'main' branch is selected, and the 'Codespaces' tab is highlighted with a red box and a red number '2'. The 'Codespaces' section shows 'No codespaces' and a button 'Create codespace on main' highlighted with a red box and a red number '3'. The file list on the left includes '.github/workflows', '.vscode', 'app', 'solutions', 'tests', and '.gitignore', all with 'Initial commit' status.

Repo — code, tests & assets

```
├── LICENSE
├── README.md
├── .gitignore
├── app
│   ├── __init__.py
│   └── hello.py
├── assets
│   └── images
│       ├── open-in-workspace.png
│       └── workspace-structure.png
├── requirements.txt
├── solutions
│   ├── package-exercise-original.yml
│   └── package.yml
└── tests
    └── test_hello.py
```

Repo — workflows

- Minimal Python template with `.github/workflows`



```
.github/workflows/  
- package-matrix.yml  
- package.yml  
- test.yml
```

- CI definitions live under `.github/workflows`
- Each YAML file defines a workflow

YAML basics



- Key: values, indentation, lists, nested structures

YAML — key/value

```
name: CI – Test & Lint
```

- Workflow name displayed in the Actions UI

PS: In YAML, strings do not need to be enclosed in quotes unless they contain special characters.

All of the following four lines are valid in YAML:

```
greeting: Hello  
greeting: Hello World  
greeting: "Hello World"  
path: "/home/user/my folder"
```

But THIS is NOT valid:

```
path: /home/user/my folder # ❌
```

YAML — nested keys

```
on:  
  push:  
  pull_request:  
    branches:  
      - main
```

- `on:` defines events that trigger workflows
- `push:` and `pull_request:` are nested under `on:` meaning they are parallel triggers

PS: the value of a key can be empty, as shown above.

As long as the key is present, the event should trigger the workflow.

YAML — lists & steps

```
branches:  
  - main  
  - develop
```

Or the following expression, they are equivalent

```
branches: [main, develop]
```

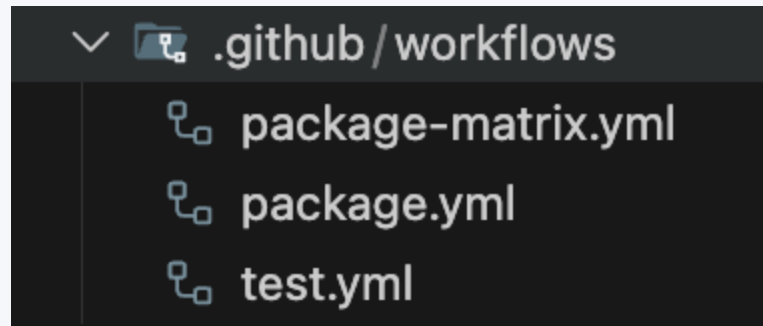
```
steps:  
  - name: Checkout Repo  
    uses: actions/checkout@v4  
  - name: Run tests  
    run: pytest -q
```

If we convert the above to JSON, it would look like this:

```
"steps": [  
  {  
    "name": "Checkout Repo",  
    "uses": "actions/checkout@v4"  
  },  
  {  
    "name": "Run tests",  
    "run": "pytest -q"  
  }  
]
```

Breakdown of test.yml

- Understanding a simple CI workflow

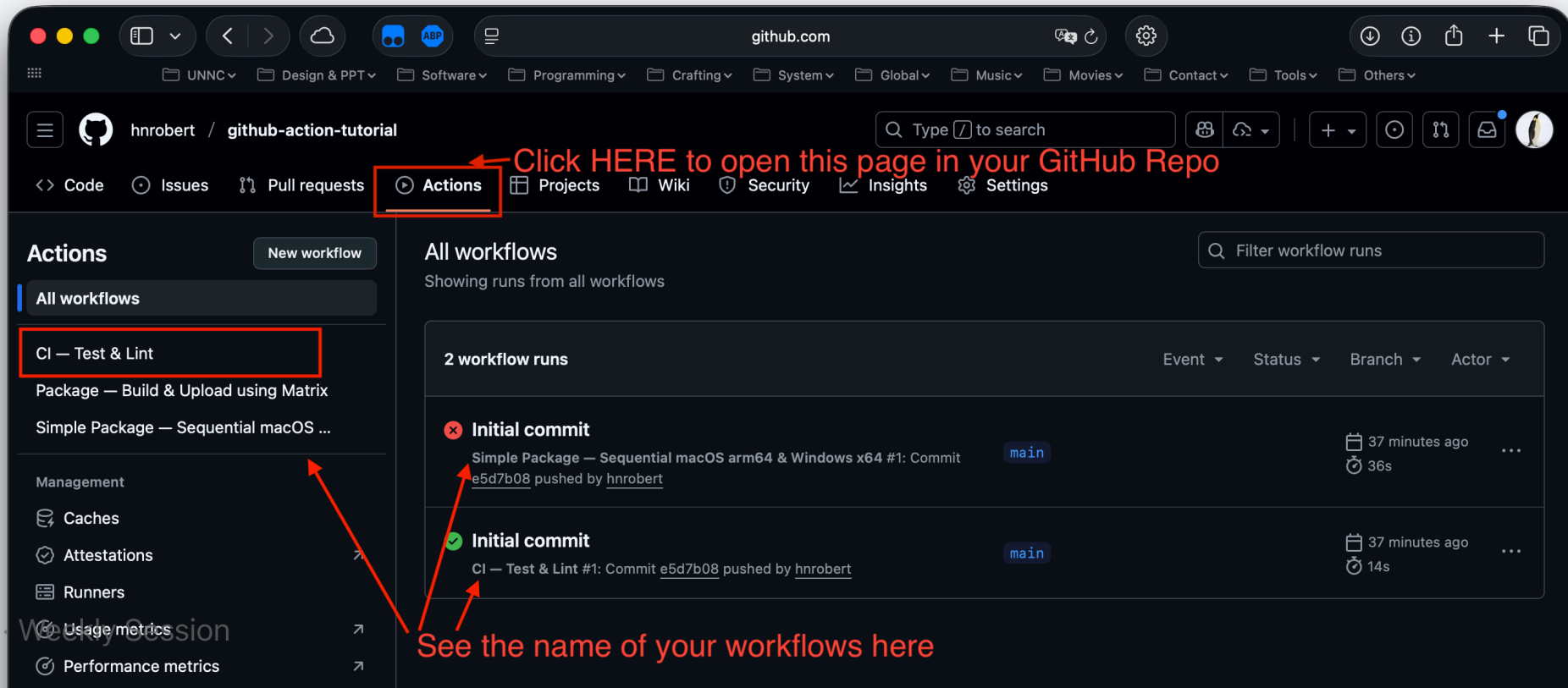


- We'll read through `test.yml` step-by-step first.

Workflow — Name

name: CI — Test & Lint

- The workflow's display name in Actions UI



Workflow — Triggers

```
on:  
  push:  
    branches: [main]  
  pull_request:  
    branches: [main]
```

- Triggers the workflow on pushes and PRs to `main`

Jobs — top-level

```
jobs:  
  test:  
    runs-on: ubuntu-latest
```

- Each job runs on a runner; jobs can run in parallel

Job — Runs-on

```
runs-on: ubuntu-latest
```

- Choose `ubuntu`, `macos`, `windows`, or self-hosted runners

Job — Steps (overview)

```
jobs:
  test:
    # ...
    steps:
      - name: Checkout Repo
        # ...
      - name: Set up Python
        # ...
      - name: Install dependencies
        # ...
      - name: Run lint
        # ...
      - name: Run tests
        # ...
```

- Steps are executed sequentially inside a job

Step: Checkout the repo

```
– name: Checkout Repo  
  uses: actions/checkout@v4
```

- Grants the job access to repository code

Step: Set up Python

```
- name: Set up Python
  uses: actions/setup-python@v4
  with:
    python-version: '3.11'
```

- Installs Python without manual commands

If you want to install python manually, you may use the following step:

```
- name: Set up Python Manually
  run: |
    sudo apt-get update
    sudo apt-get install -y python3.11 python3.11-venv python3.11-dev
```

That's why we use `actions/setup-python`, simple and stable.

Step: Install dependencies

```
- name: Install dependencies
  run: |
    python -m pip install --upgrade pip
    pip install -r requirements.txt
```

- Ensures library dependencies are available

Step: Run linting

```
- name: Run autopep8 check  
  run: |  
    pip install autopep8  
    autopep8 --diff --recursive app tests
```

- Keeps code style consistent

Step: Run tests

```
- name: Run tests
  env:
    PYTHONPATH: ${github.workspace}
  run: pytest -q
```

- Executes test suite and reports failures
- `PYTHONPATH` environment variable ensures `app/` is importable

How to package this project locally (for reference)

Usually we would build a standalone executable for such small project with PyInstaller.

On macOS/Linux:

```
python3 -m venv .venv
source .venv/bin/activate
.venv/bin/pip install -r requirements.txt
.venv/bin/pyinstaller --onefile app/hello.py
ls -la dist
```

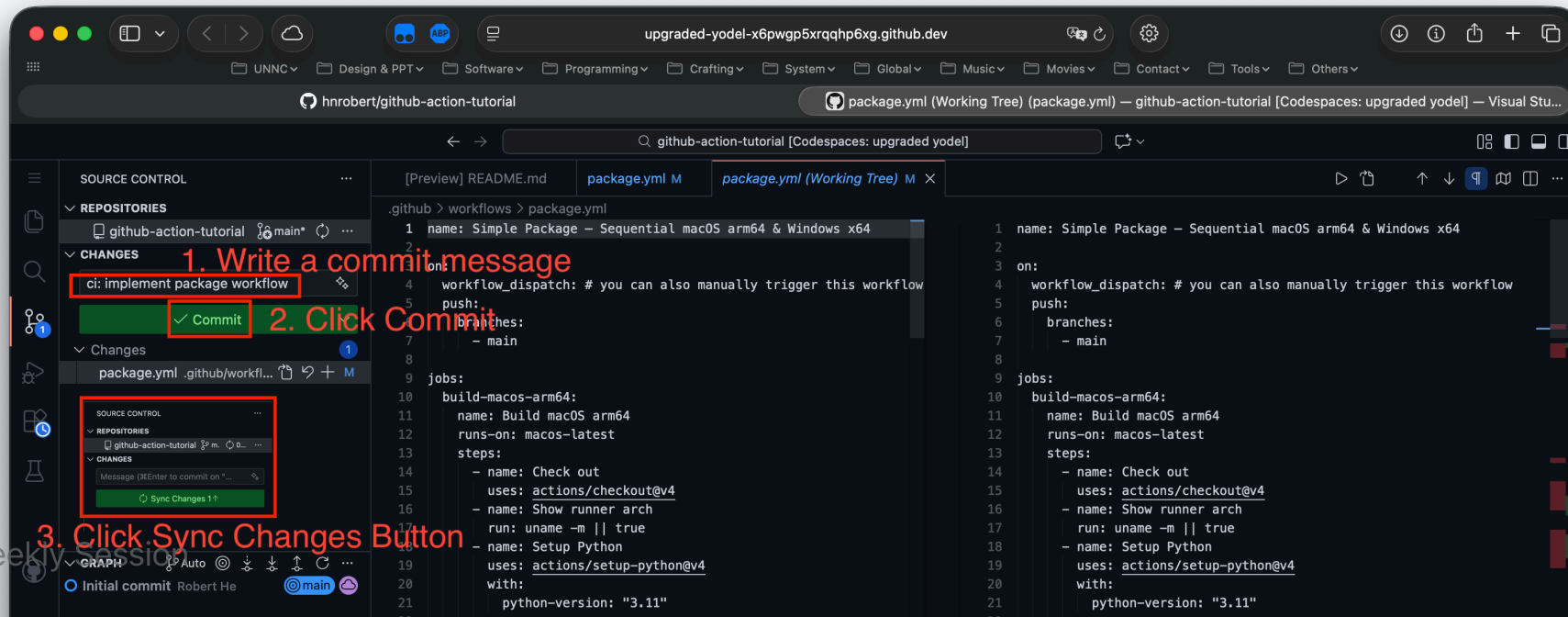
On Windows PowerShell:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
.venv\Scripts\pip.exe install -r requirements.txt
.venv\Scripts\pyinstaller.exe --onefile app/hello.py
dir dist
```


Exercise

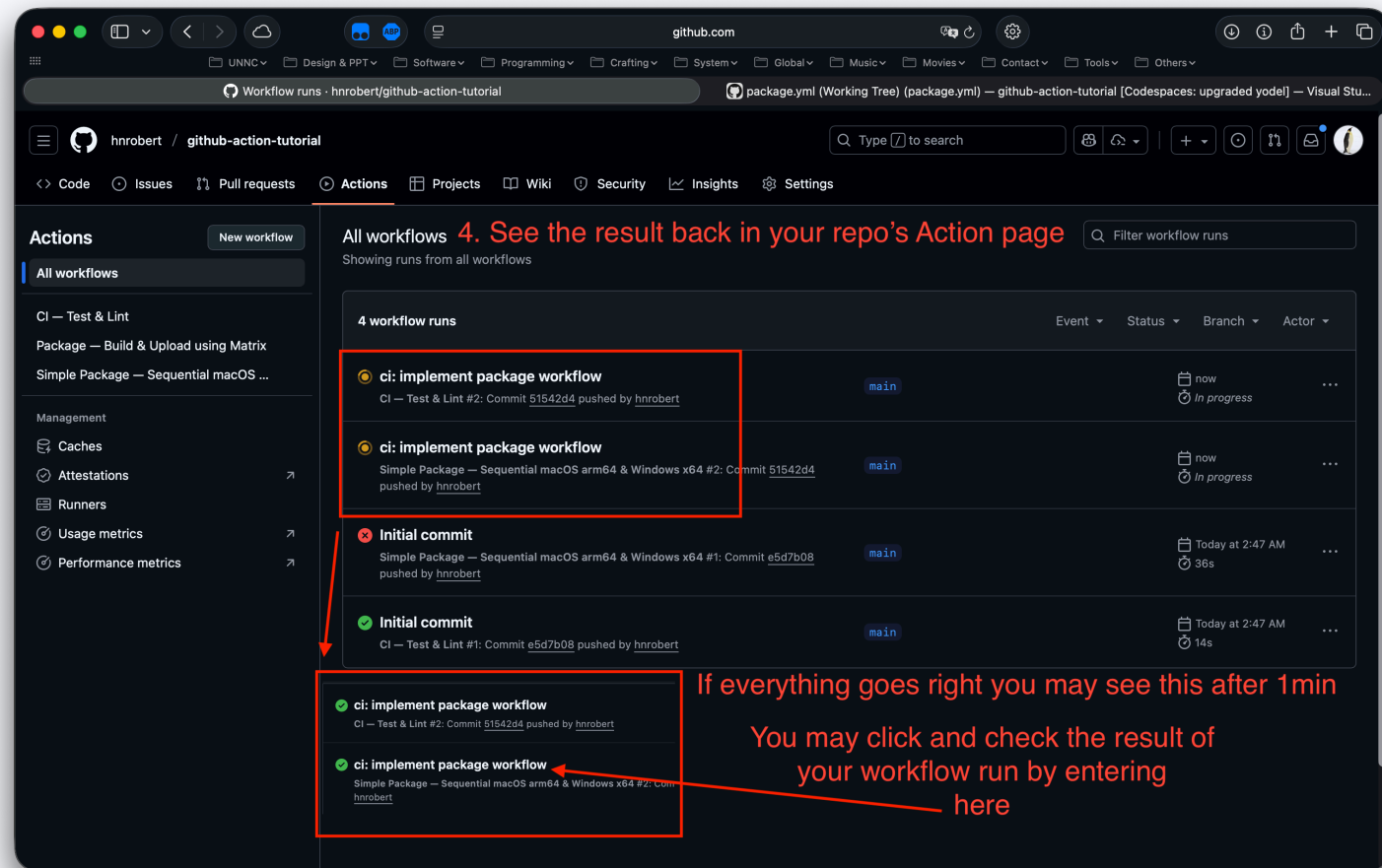
But to upload your build artifacts every time to GitHub from local would be tedious, so we often automate this with GitHub Actions.

- Implement package.yml
- Commit your workflow updates & push (sync) to trigger the workflow run

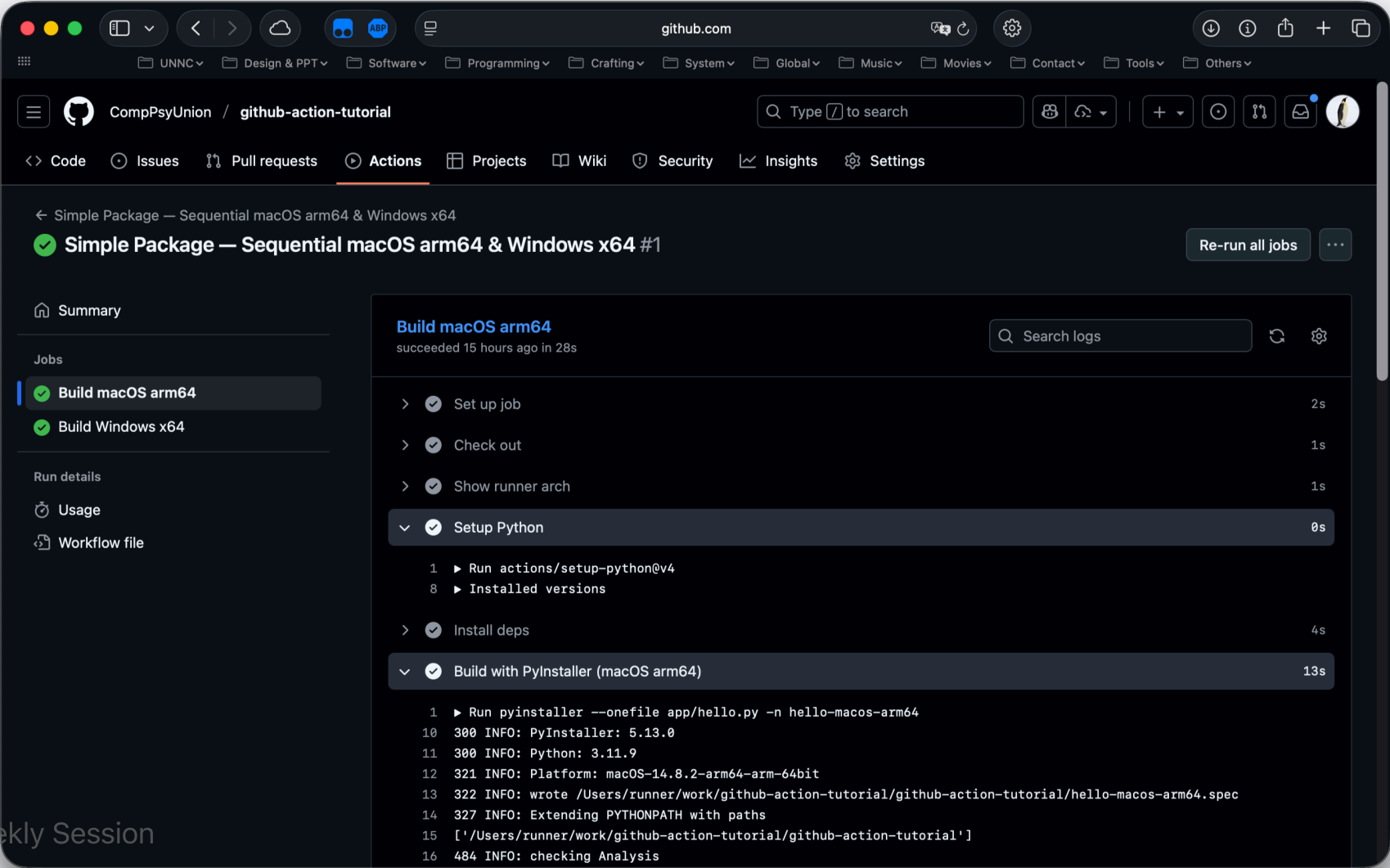


Watch workflow running

- Jobs show logs and status in Actions tab



Watch workflow running - details



Packaging exercise — result

- Artifacts built & uploaded automatically

The screenshot shows a GitHub Actions workflow run for the repository 'hnrbert / github-action-tutorial'. The workflow is named 'ci: implement package workflow #2' and has a status of 'Success'. It was triggered by a push to the 'main' branch. The total duration is 1m 47s, and there are 2 artifacts.

The workflow file 'package.yml' is shown, with two jobs: 'Build macOS arm64' (1m 43s) and 'Build Windows x64' (1m 15s).

The 'Artifacts' section shows two artifacts produced during runtime:

Name	Size	Digest
hello-macos-arm64	7.29 MB	sha256:50ce99dfe989d754141f003c7f999da768b8a70f10e751b20...
hello-windows-x64	8.03 MB	sha256:1d4099db7c06aae0add8874d3e849ed1576e0335ade5e6268...

Red text overlay: "You may see your packaged programme here Try download one of them that matches the platform of your laptop, you may also run it and verify"

More job features

- `strategy` and `needs` help with matrices and dependencies

```
strategy:  
  matrix:  
    os: [ubuntu-latest, macos-latest, windows-latest]  
needs: [build]
```

- Define environment variables for steps

```
env:  
  PACKAGE_NAME: hello
```

- Use `if:` to conditionally run steps or jobs

```
if: github.event_name == 'push'
```

- Use `outputs` to pass data between jobs

```
outputs:  
  package-path: ${ steps.package.outputs.path }
```

- Use `secrets` to store sensitive data

```
secrets:  
  API_KEY: ${ secrets.API_KEY }
```

- Use `timeout-minutes` to limit job duration

```
timeout-minutes: 10
```

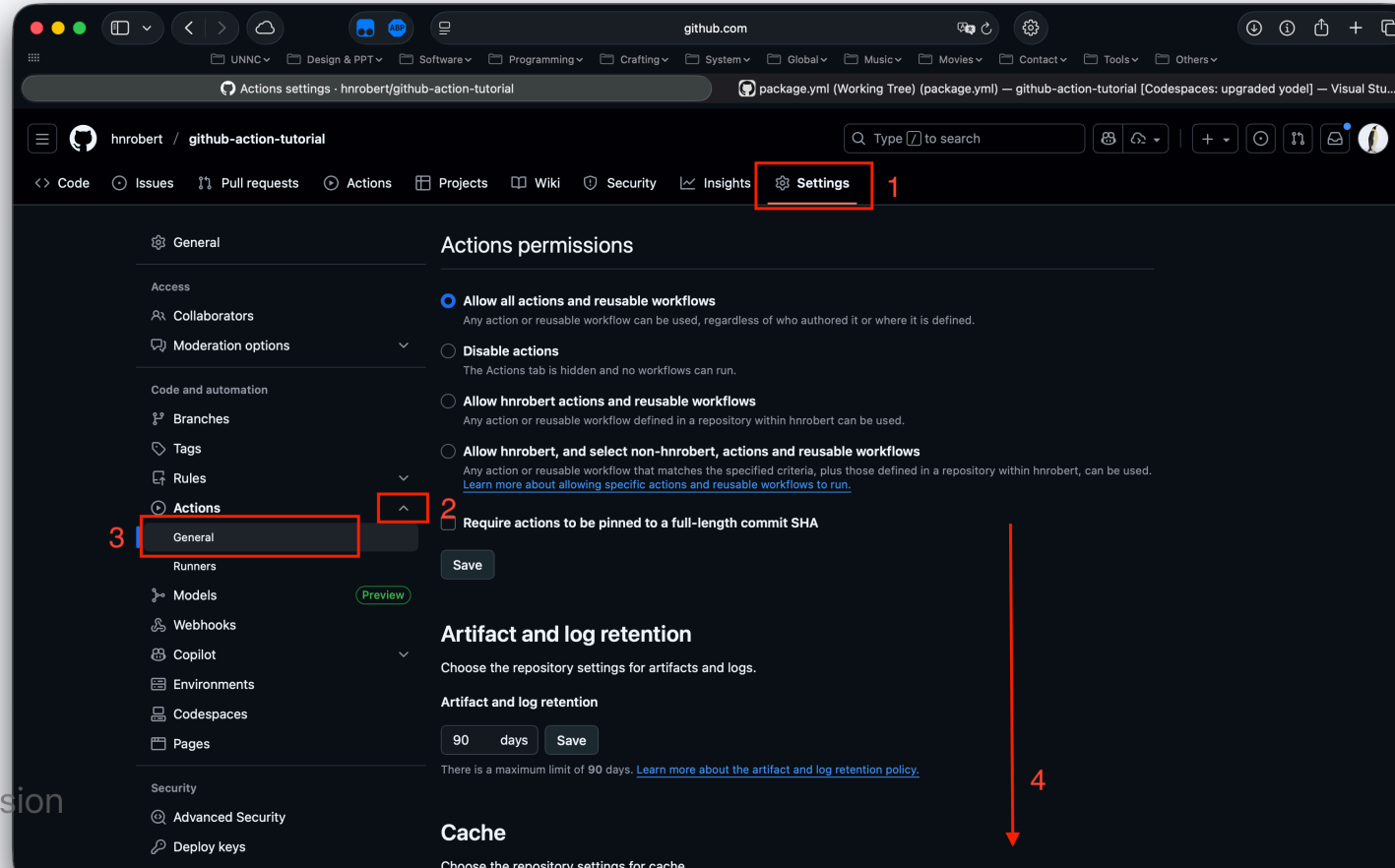
Release flow (automatic packaging and upload assets to release)

The repository includes a workflow that automatically builds packages for multiple OS/Python combinations and attaches the build artifacts to a GitHub Release.

- `package-matrix.yml` — runs packaging on `ubuntu`, `macos`, and `windows` using a matrix and attaches the build artifacts to the release when the workflow is triggered by the `release` event.

Release permission — Settings

- Wait, your workflow need some permissions to upload assets to release
- Grant "Read and write" to allow uploading release assets



Release permission — Set workflow permissions

- Choose "Read and write permissions" and save

Workflow permissions

Choose the default permissions granted to the GITHUB_TOKEN when running workflows in this repository. You can specify more granular permissions in the workflow using YAML. [Learn more about managing permissions.](#)

☒ **Read and write permissions**

Workflows have read and write permissions in the repository for all scopes.

5

☐ **Read repository contents and packages permissions**

Workflows have read permissions in the repository for the contents and packages scopes only.

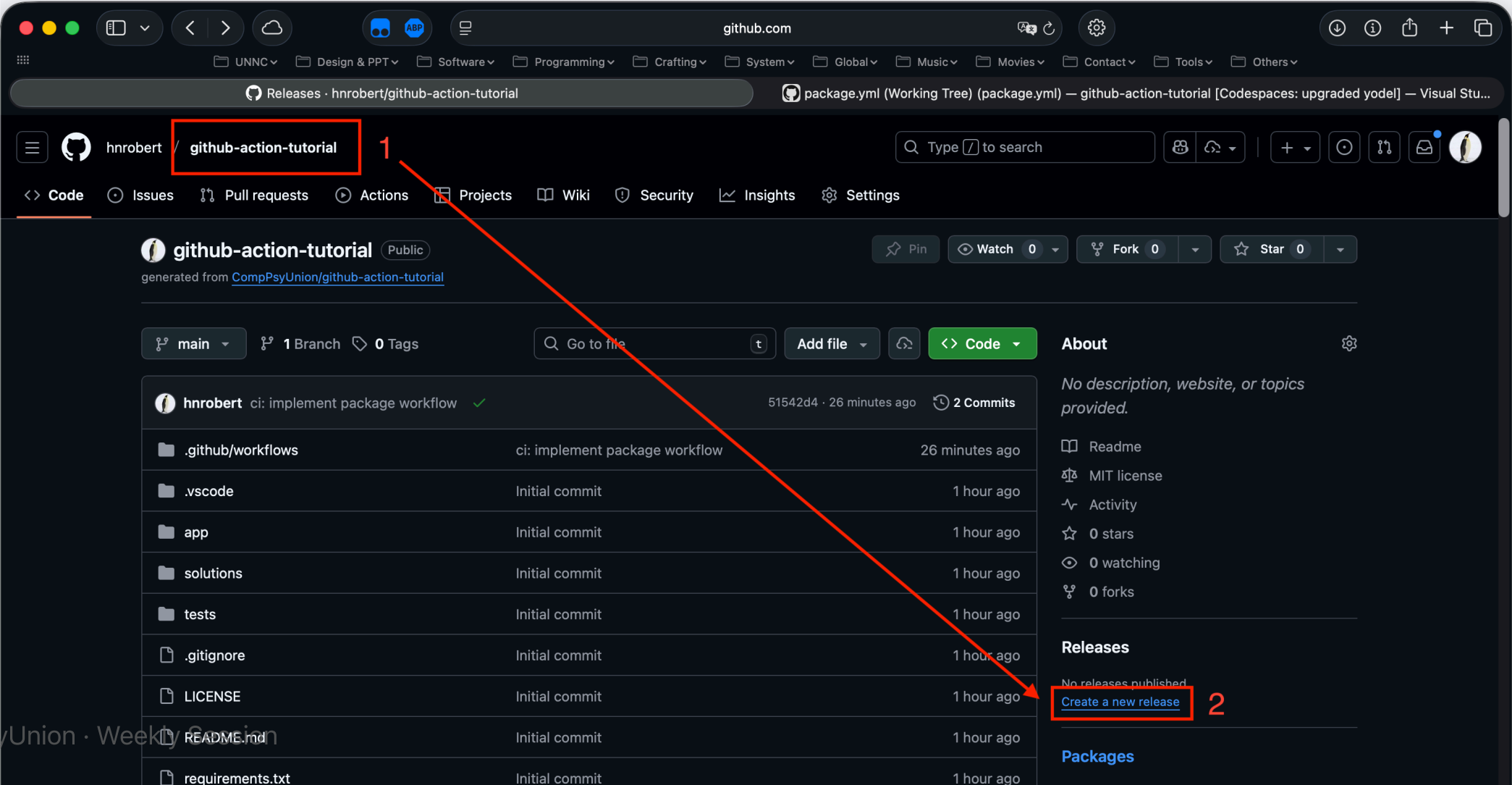
Choose whether GitHub Actions can create pull requests or submit approving pull request reviews.

☐ **Allow GitHub Actions to create and approve pull requests**

Save

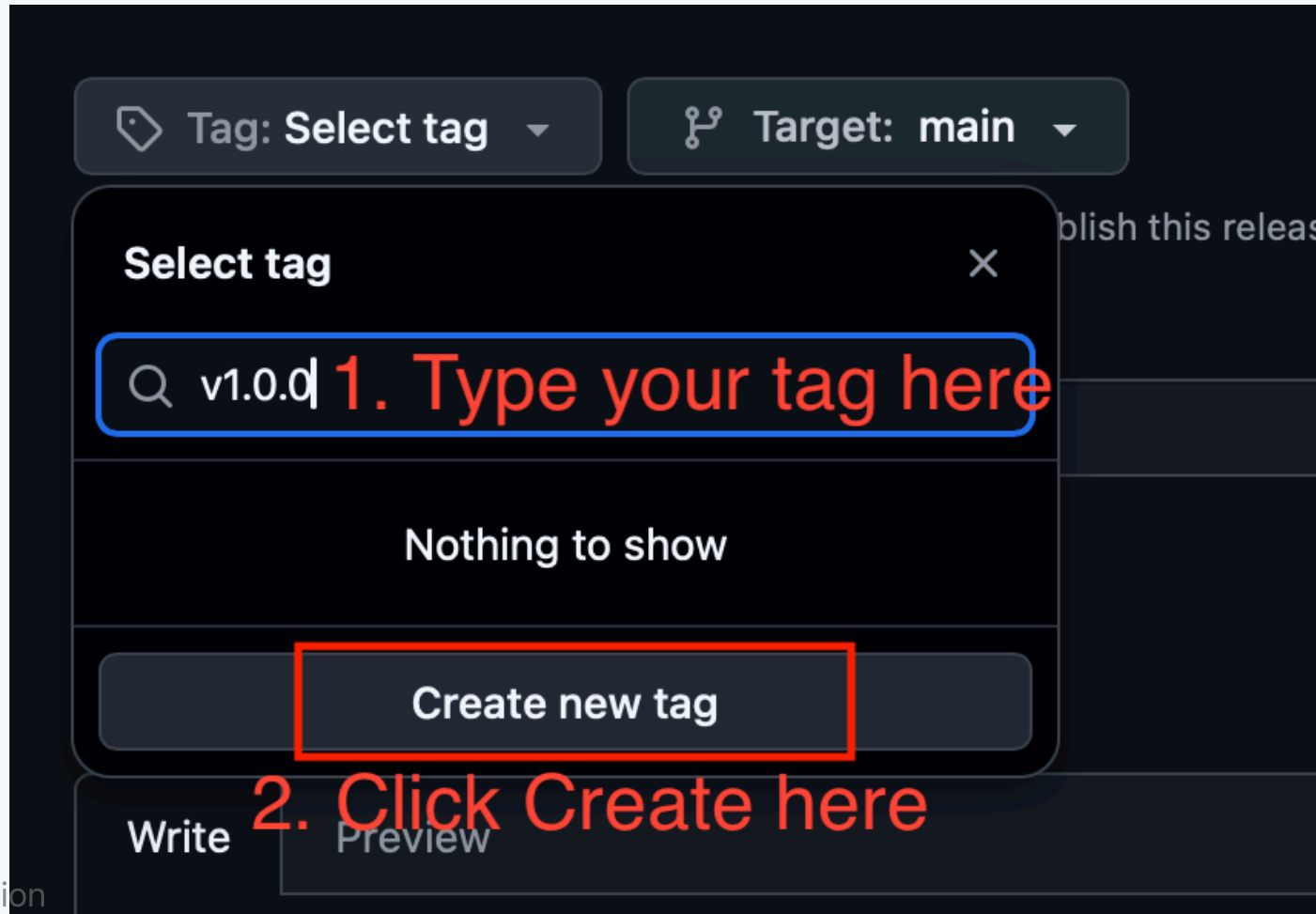
6

Create Release — Start



Create Release — Tag & Title

- Add tag (e.g., `v1.0.0`) and title



The screenshot shows the 'Create Release' dialog in GitHub. At the top, there are two dropdown menus: 'Tag: Select tag' and 'Target: main'. Below these is a 'Select tag' modal window. Inside this modal, a search input field contains 'v1.0.0' and is highlighted with a blue border. A red text annotation '1. Type your tag here' points to this input field. Below the input field, the text 'Nothing to show' is displayed. At the bottom of the modal, a button labeled 'Create new tag' is highlighted with a red border. A red text annotation '2. Click Create here' points to this button. The background of the dialog is dark, and the text is light gray.

Tag: Select tag Target: main

Select tag

Search v1.0.0

Nothing to show

Create new tag

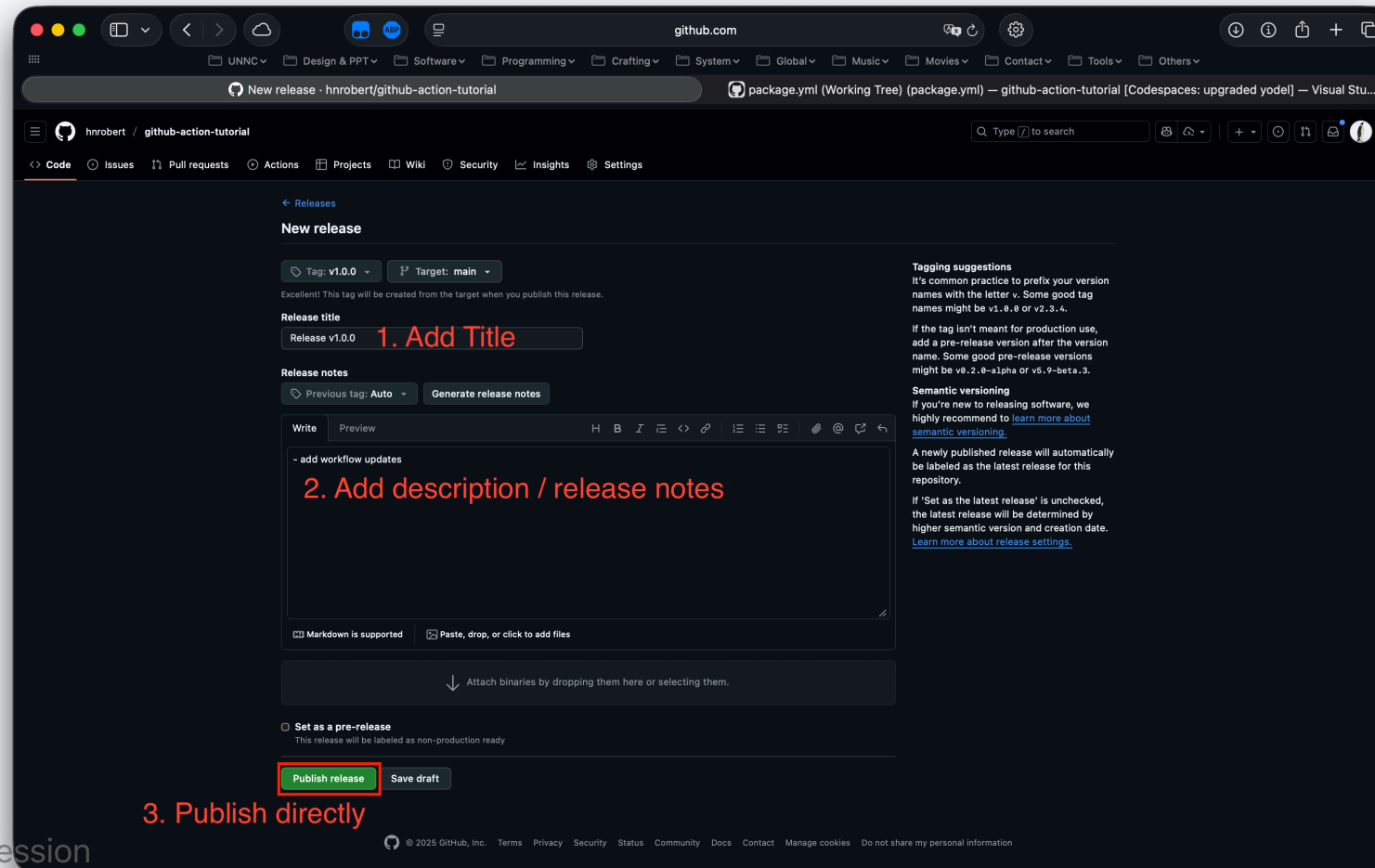
Write Preview

1. Type your tag here

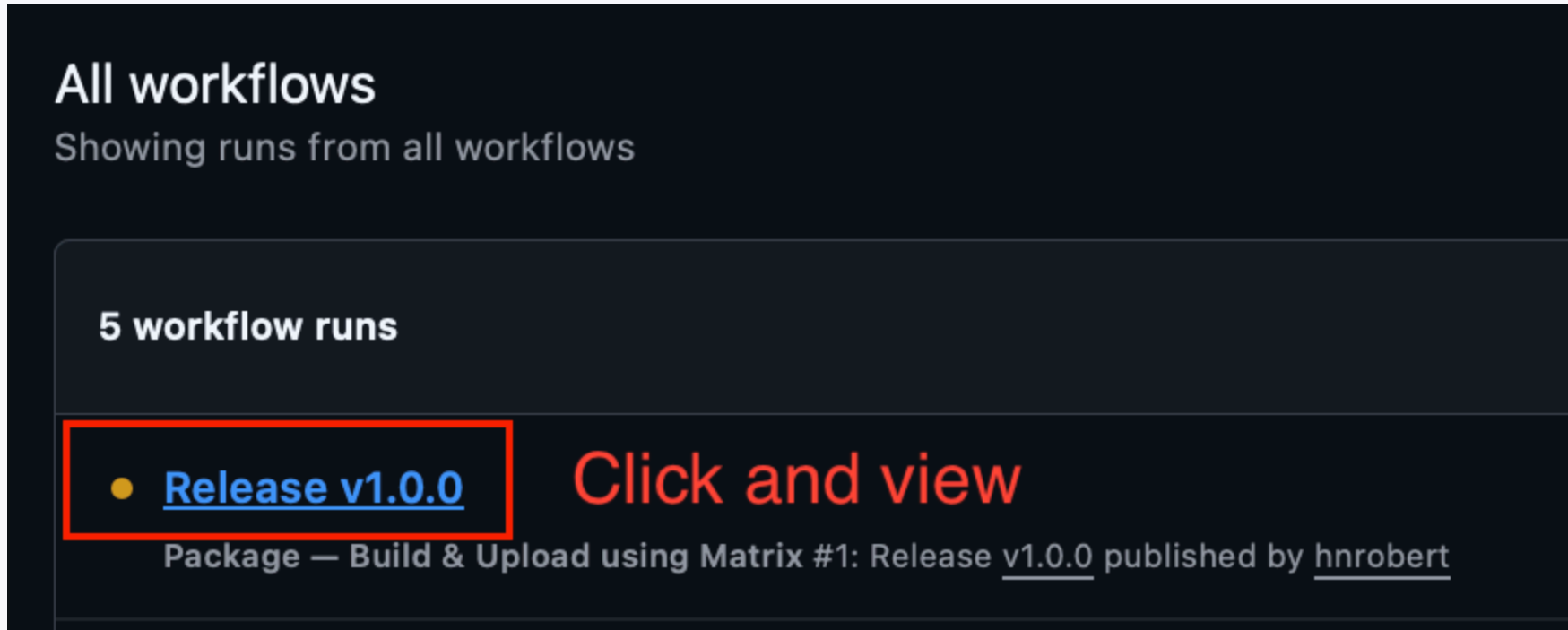
2. Click Create here

Publish Release

- Use the tag and add a title & description, click to publish



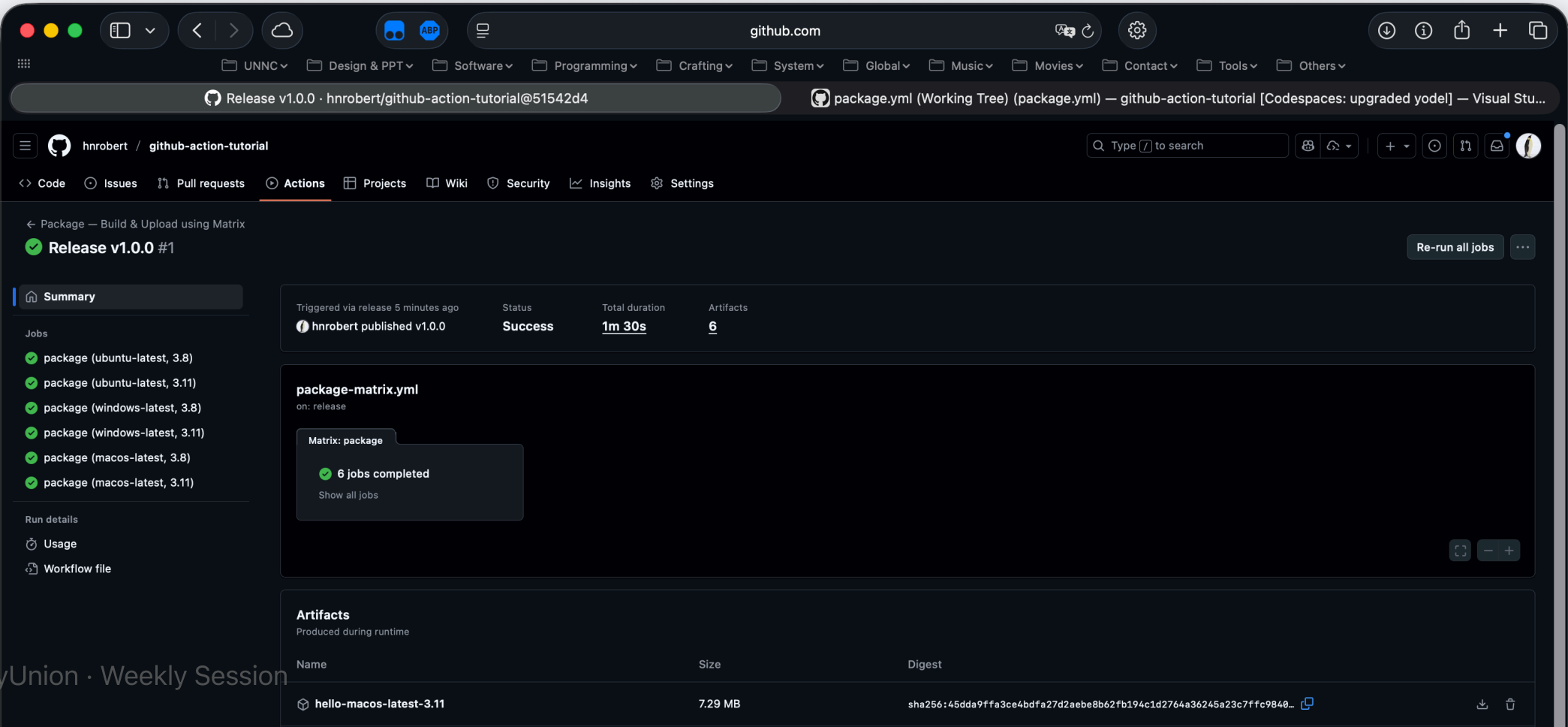
Auto-build



- Click the Release title in Actions tab to see build progress

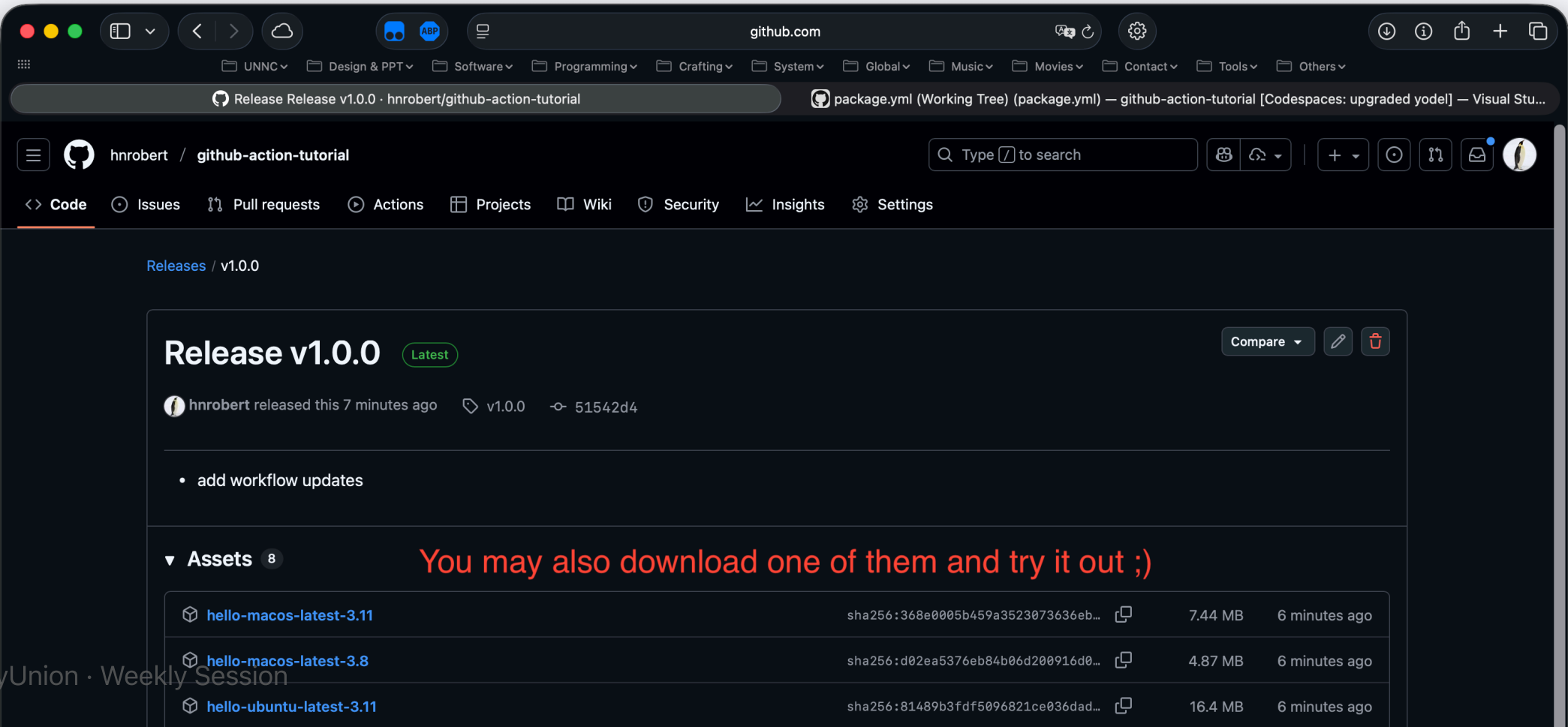
Matrix success & artifacts

- Check artifacts and success across OSes



Release Page & Artifacts

- Build artifacts are attached to the release



Best Practices

- Keep CI jobs small
- Cache dependencies for speed
- Use marketplace actions for common tasks

Popular Actions

- `actions/checkout` — checkout the repository
- `actions/setup-python` — set up Python
- `actions/setup-node` — set up Node.js
- `actions/setup-go` — set up Go
- `actions/setup-dotnet` — set up .NET
- `actions/upload-artifact` — upload artifacts
- `actions/cache` — cache dependencies
- `pytest` and `autopep8` for testing and automatic formatting

Useful Link

- GitHub Actions docs: <https://docs.github.com/en/actions>

Q&A ;)

Any questions so far?

Ask what you want to do with GitHub Actions!

The End

Thank you!